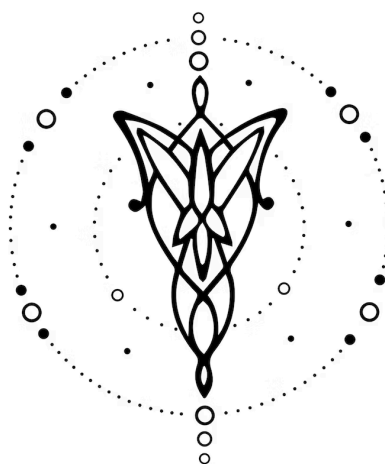




HCI

HTML TO COMPOSABLE INSTRUCTIONS

v1.0.0

1. Equipo	1
2. Repositorio	1
3. Dominio	2
4. Construcciones	2
5. Casos de Prueba	3
6. Ejemplos	3

1. Equipo

Nombre	Apellido	Legajo	E-mail
Matias	Cuneo Gima	64492	mcuneogima@itba.edu.ar

2. Repositorio

La solución y su documentación serán versionadas en: [HCI](#).

3. Dominio

Desarrollar un compilador que permite transformar archivos HTML (HyperText Markup Language) a funciones @Composable en Kotlin.

Para reducir la complejidad del proyecto, se limitó la lista de etiquetas HTML aceptadas a un subconjunto de las mismas.

La implementación satisfactoria de este lenguaje permitiría pasar archivos HTML a funciones @Composable, las cuales son los bloques fundamentales de la estructura y estilo de una aplicación Android nativa. Esto con el objetivo de que los desarrolladores que tengan código HTML (posiblemente de desarrollar aplicaciones web) puedan, fácilmente, migrarlo a una arquitectura Android preservando la estructura.

4. Construcciones

El lenguaje desarrollado debería ofrecer las siguientes construcciones, prestaciones y funcionalidades:

- (I). Se aceptan las etiquetas: h1 a h6, p, button, ul y li, div, input e img.
- (II). Se aceptan en formato <tag></tag> y <tag/>.
- (III). Los tags header y footer generan componentes reutilizables.
- (IV). Se ignoran etiquetas y atributos relacionados a estilo y comportamiento.
- (V). Ciertas combinaciones de etiquetas forman componentes.

Estas son:

- <div> <h2/> <p/> </div> a CustomCard
- <div> <p/> <input/> </div> a LabeledInput
- <div> <p/> </div> a ImageWithFooter

5. Casos de Prueba

Se proponen los siguientes casos iniciales de prueba de **aceptación**:

- (I). Un archivo que tenga hermanos simples (


```
<h1>SOY JUAN</h1>
<p>Hola mi nombre es Juan y como sandwiches de miga</p>
<button>Clickeame</button>
```

).
- (II). Un archivo que tenga un div con hijos.
- (III). Un archivo que tenga una lista sin anidar.
- (IV). Un archivo con Self-closing y forma larga mezcladas.
- (V). Un archivo con h1 a h6.
- (VI). Un archivo con un button dentro de un div.
- (VII). Un archivo con una lista dentro de un div.
- (VIII). Un archivo que produzca un componente.
- (IX). Un archivo con etiquetas sin contenido.
- (X). Un archivo con comentarios y head.

Además, los siguientes casos de prueba de **rechazo**:

- (I). Un archivo malformado por etiquetas abiertas.
- (II). Un archivo malformado por etiquetas no toleradas.
- (III). Un archivo malformado por atributos no tolerados.
- (IV). Un archivo que falle en seguir la estructura estándar de HTML
(`<html>`
`<head/>` (Opcional y no pusheado)
`<body>`
`...`
`</body>`
`</html>`).
- (V). Un archivo con `li` fuera de `ul`.

6. Ejemplos

A continuación, el código 1.a y 1.b siendo la entrada y salida a partir de un archivo conteniendo una estructura no anidada:

```
<html>
<body>
<h1>SOY JUAN</h1>
<p>Hola mi nombre es Juan y como sandwiches de miga</p>
<button>Clickeame</button>
</body>
</html>
```

Código 1.a: Archivo HTML con estructura anidada.

```
@Composable
fun output(){
    Column(){
        Text( text="SOY JUAN", fontSize= 20.sp )
        Text ( text="Hola mi nombre es Juan y como sandwiches de miga",
            fontSize= 4.sp )
        Row( modifier = Modifier.clickable {} ){
            Text( text="Clickeame")
        }
    }
}
```

Código 1.b: Archivo Kotlin de salida de una estructura anidada.

Entrada y salida (códigos 2.a y 2.b) a partir de un archivo conteniendo una estructura anidada y una etiqueta sin contenido:

```
<html>
<body>
<div>
  <input type=text/>
  <button> Confirmar </button>
</div>
<p>Esto es un parrafo</p>
</body>
</html>
```

Código 2.a: Archivo HTML con estructura anidada y etiqueta vacía.

```
@Composable
fun output(){
  Column(){
    Row(){
      TextField( keyboardOptions = KeyboardOptions(keyboardType =
                                                KeyboardType.Number),
                placeholder = { Text(text = "" ) }
            )

      Row( modifier = Modifier.clickable {} ){
        Text( text="Clickeame")
      }
    }
    Text ( text="Esto es un parrafo",
          fontSize= 4.sp )
  }
}
```

Código 2.b: Archivo Kotlin de salida de una estructura anidada.

Por último, se tiene el código 3.a y 3.b correspondiendo entrada y salida a partir de un archivo conteniendo una estructura anidada y pasado a componentes:

```
<html>
<body>
<header>
  <h1>Soy el header</h1>
</header>
<div>
```

```
<input type=text/>
<button> Confirmar </button>
</div>
<p>Esto es un parrafo</p>
</body>
</html>
```

Código 3.a: Archivo HTML con estructura anidada y conteniendo un componente.

```
@Composable
fun Header(){
    Row(){
        Text( text="Soy el header", fontSize= 20.sp )
    }
}

@Composable
fun output(){
    Column(){
        Header()
        Row(){
            TextField( keyboardOptions = KeyboardOptions(keyboardType =
                                                                    KeyboardType.Number),
                        placeholder = { Text(text = "" ) }
                    )

            Row( modifier = Modifier.clickable {} ){
                Text( text="Clickeame")
            }
        }
        Text ( text="Esto es un parrafo",
                fontSize= 4.sp )
    }
}
```

Código 3.b: Archivo Kotlin de salida de una estructura anidada y pasado a componentes.

7. Documentación

Finalmente, se incluye la documentación relevante al contenido de la especificación.

Android : <https://developer.android.com/develop?hl=es-419>

Kotlin : <https://kotlinlang.org/docs/home.html>

Jetpack Compose :

<https://developer.android.com/develop/ui/compose/documentation?hl=es-419>